



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

**Dept. of Computer Science
Faculty of Science and Technology**

CSC 2210: OBJECT ORIENTED PROGRAMMING 2

Fall 2025-2026

Section: L

Group No: 9

**Project Report On
Rental Management System
Supervised By
KAZI ASIF AHMED
Submitted By:**

Name	ID
1. MD.Naimul Haque Tashin	23-55346-3
2. Mohammad Shihab Hossain	24-56547-1
3. Shanjihan jaman Shuchi	24-56543-1

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				Total =
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.				

Table of Contents

1. INTRODUCTION	3
1.1 INTRODUCTION SubPart.....	4
2. ER DIAGRAM	9
2.1 Normalization	10
1. Relation Name: User Owns Property.....	10
2. Relation Name: User Makes Booking	10
3. Relation Name: Property Has Booking.....	11
4. Relation Name: Booking Paid By Payment.....	11
5. Relation Name: User Makes Payment	12
6. Relation Name: User Writes Review	13
7. Relation Name: Property Has Review	13
8. Relation Name: Property Contains Images.....	14
9. Relation Name: User Sends Message to User (M-M)	14
2.2 Finalization (Final Tables).....	15
3. UML DIAGRAM.....	16
3.1 Use Case Diagram.....	16
3.2 Activity Diagram.	17
6.CONCLUSION.....	23

1.INTRODUCTION

- **Rental Management System**

The **Rental Management System** is a desktop-based application developed using the **C# .NET framework** to simplify and modernize the process of managing house rentals. By replacing traditional paperwork with a centralized digital platform, the system enables tenants, landlords, and administrators to efficiently manage property listings, bookings, payments, and communication.

- **User Interface & Architecture**

To deliver a polished and professional user experience, the application is built using **WinForms** combined with the **AntdUI** library, providing a modern and visually appealing interface.

The system follows a **Layered Architecture**, ensuring a clear separation between:

- **Business Logic Layer (BLL)** – handles core application rules and workflows
- **Data Access Layer (DAL)** – manages database interactions

This structure improves maintainability, scalability, and ease of future updates.

- **User Roles**

The system is designed around three primary user roles:

Tenants

- Browse available rental properties
- Filter listings by city, price, and availability
- Book properties and communicate directly with landlords

Landlords

- Manage and list rental properties
- View dashboards displaying total earnings
- Approve or reject tenant booking requests

SuperAdmins

- Manage all users and system data
- Monitor transactions and overall system activity
- **Key Features & Security**

Smart Database Management

- Powered by **SQL Server** to store user profiles, property data, bookings, transactions, and chat history

Security-Focused Design

- Passwords are protected using **BCrypt hashing**, ensuring they are never stored in plain text
- Custom session management restricts access based on user authorization

Seamless Payment Simulation

- Supports simulated integration with payment gateways such as **SSLCommerz**
- Enables mobile banking payments via platforms like **Bkash** and **Nagad**, with landlord verification

Reliable Notifications

- Integrated with the **Resend API** to send transactional emails such as account verification and system alerts

1.1 INTRODUCTION SubPart

Technology Stack: - **Language:** C# (.NET Core / Framework) - **UI Framework:** Windows Forms (WinForms) with AntdUI for modern styling. - **Database:** Microsoft SQL Server (LocalDB / SQLEXPRESS). - **Architecture:** Layered Architecture (UI -> Service -> Repository -> Database). - **Authentication:** Custom Session Management with BCrypt Password Hashing. - **External APIs:** Resend API is used for transactional email delivery and SMTP-based verification, including account verification, notifications, and system alerts.

Purpose:

A desktop application to manage house rentals. It allows **Tenants** to browse and book properties, **Landlords** to list properties and manage bookings, and **SuperAdmins** to oversee users and transactions.

- **2. System Architecture**

The project follows a **Layered Architecture** with **Service-Repository Pattern** characteristics.

- **High-Level Layers:**

2.1 Presentation Layer (UI):

- **Forms:** Form1 (Login), UserDashboard (Tenant), HomeownerDashboard (Landlord), AdminDashboard (SuperAdmin).
- **Components:** Reusable sections like DashboardSections and AdminDashboardSections.
- **Controls:** Uses standard WinForms controls + AntdUI library for buttons, tables, and messages.

2.2 Business Logic Layer (Services):

- Contains the “Brain” of the application.
- Located in Services/ folder (e.g., AuthService, PropertyService, TenantService).
- Validates rules (e.g., “End date must be after Start date”, “Only Tenants can book”).

2.3 Data Access Layer (Data / Repositories):

- Handles raw SQL connections using ADO.NET (SqlConnection, SqlCommand).
- Located in Data/ folder (e.g., UserRepository, Database.cs).
- Executes CRUD operations mostly via direct interaction or helper methods.

2.4 Database:

- SQL Server Relational Database (HomeRentalDB).

- **3. Folder Structure & Responsibilities**

Folder	Responsibility
Forms	Contains all Windows Forms (Screens). Form1.cs is the entry point (Login). Subfolders organize dashboard sections.
Services	Business logic classes. They call Repositories/Database and are called by Forms.

Folder	Responsibility
Data	Database connection logic (Database.cs), repositories (UserRepository.cs), and initialization (DatabaseInitializer.cs).
Models	POCO (Plain Old CLR Objects) classes representing DB tables (e.g., User, PropertyModel).
Assets	Stores images and icons used in the UI (menu.png, logo.png).
db	SQL Scripts for table creation (create_table.sql) and data seeding.
Helpers	Utility classes like EmailHelper for OTPs.

- **4. Database Design (Schema)**

- **Key Tables & Relationships**

4.1 USERS

- **Purpose:** Stores all users (Tenant, Landlord, SuperAdmin).
- **Columns:** UserID (PK), Email, PasswordHash, UserType, Phone, IsActive.
- **Key Logic:** UserType determines if they see Tenant or Landlord dashboard.

4.2 PROPERTIES

- **Purpose:** Stores rental listings.
- **Columns:** PropertyID (PK), LandlordID (FK -> USERS), RentAmount, Status ('Available', 'Rented'), Address.
- **Relationship:** One Landlord has Many Properties.

4.3 BOOKINGS

- **Purpose:** Tracks rental agreements.
- **Columns:** BookingID, PropertyID (FK), TenantID (FK -> USERS), Status ('Pending', 'Approved'), StartDate, TotalAmount.
- **Relationship:** One Property can have Many Bookings (historically), but usually one active.

5 PAYMENTS

- **Purpose:** Records financial transactions.
- **Columns:** PaymentID, BookingID (FK), Amount, TransactionID, Method ('Bkash', etc.), Status.
- **Relationship:** One Booking has Many Payments.

6 MESSAGES

- **Purpose:** Chat between Tenant and Landlord.
- **Columns:** SenderID, ReceiverID, PropertyID, Content, IsRead.

7 PROPERTY_IMAGES

- **Purpose:** Multi-image support for properties.
- **Columns:** ImageID, PropertyID (FK), ImagePath.
- **Relationship:** One Property has Many Images.

5. Feature List

5.1 Key Workflows

5.2 Booking Flow

1. **Search:** Tenant searches in RentAllSearch. Defaults to “Available” properties.
2. **View:** Tenant clicks a property card -> PropertyDetails opens.
3. **Request:** Tenant selects dates and clicks “Book Now”.
 - System creates a row in BOOKINGS with status 'Pending'.
4. **Approval:** Landlord sees request in RequestList.
 - Click **Approve** -> updates BOOKINGS status to 'Approved'.
 - Updates PROPERTIES status to 'Rented'.

• 5.2 Payment Flow

1. **Initiation:** In TenantPaymentList, Tenant sees “Pay Now” for approved bookings.
2. **Form:** Opens Payment.cs. Tenant enters Amount and Transaction ID.
3. **Processing:**
 - Code validates inputs.
 - Calls TenantService.CreatePaymentForBooking.
 - Updates PAYMENTS table.
4. **Verification:** Landlord sees payment in PaymentList.

5.3 Authentication & Security

- **Hashing:** Passwords are **never** stored in plain text. Uses BCrypt.Net.BCrypt.HashPassword().
- **Session:** AppSession static class holds the logged-in user object globally.
- **Role-Based Access:**
 - Form1 routes based on UserType.
 - PropertyService.AddProperty checks if current user is ‘Landlord’.
 - PropertyDetails “Book Now” disabled for non-Tenants.

• 6. CRUD Operations

Operation	Implementation Class	SQL Command Example
Create	PropertyService.AddProperty	INSERT INTO PROPERTIES (...) VALUES (...)
Read	PropertyService.GetPropertiesByLandlord	SELECT * FROM PROPERTIES WHERE LandlordID=@id
Update	PropertyService.UpdateProperty	UPDATE PROPERTIES SET RentAmount=@amt WHERE PropertyID=@id
Delete	PropertyService.DeleteProperty	DELETE FROM PROPERTIES WHERE PropertyID=@id

Data Access Strategy: - Connection String is loaded from .env file first, then Environment Variables. - Uses using (SqlConnection ...) blocks to ensure connections are closed automatically (Preventing memory leaks).

• 7. Dashboard Statistics

Calculated in LandlordService or AdminService using aggregate SQL queries: - **Total Properties:** SELECT COUNT(*) FROM PROPERTIES WHERE LandlordID = @id - **Total Earnings:** SELECT SUM(Amount) FROM PAYMENTS p JOIN BOOKINGS b ON

p.BookingID = b.BookingID JOIN PROPERTIES prop ON b.PropertyID = prop.PropertyID
WHERE prop.LandlordID = @id - **Active Tenants**: SELECT COUNT(DISTINCT TenantID)
FROM BOOKINGS WHERE Status = 'Approved'

8. Common Debugging & Extension

How to Debug: - **Startup Crash**: Check .env file presence. Ensure db/create_table.sql ran correctly. - **Login Fails**: Check USERS table in DB. Check admin_hash.txt for generated hash match. - **Database Error**: Look at DatabaseState.cs or Console.WriteLine output in Debug window.

How to Add a New Feature (e.g., “Favorite Properties”): 1. **Database**: Create FAVORITES table (UserID, PropertyID). 2. **Model**: Create FavoriteModel.cs. 3. **Repository**: Add AddFavorite(uid, pid) in TenantRepository. 4. **UI**: Add “Heart” icon in PropertyDetails and call the repository method on click.

2.Feature List

- **UI Forms Breakdown**
- **5.1 Login & Registration (Form1.cs)**
- **Purpose**: Entry point. Handles Login, Signup, and Password Reset.
- **Panels**: pnlLogin (Default), pnlSignup (Hidden).
- **Logic**:
 - **Login**: Calls AuthService.Login(). If success, checks UserType -> Opens AdminDashboard, HomeownerDashboard, or UserDashboard.
 - **Signup**: Validates inputs -> Sends OTP (via EmailHelper mock/real) -> Opens VerifyForm -> If OTP correct, calls AuthService.Register().
- **1. Tenant Dashboard (UserDashboard.cs)**
- **Sidebar Menu**:
 1. **Browse Homes**: Loads RentAllSearch. Search bar filters by City/Rent/Amenities.
 2. **My Home**: Loads MyRentals. Shows active rented property.
 3. **My Bookings**: Loads TenantRequestsList. Shows pending/rejected requests.
 4. **Messages**: Loads MessagesSection. Chat with landlord.
 5. **Payments**: Loads TenantPaymentList. History of payments.
- **Key Logic**: Uses NavigateTo(Form) to switch content inside the main panel.
- **2. Landlord Dashboard (HomeownerDashboard.cs)**
- **Sidebar Menu**:
 1. **Dashboard**: Stats (Total Income, Total Properties).
 2. **My Properties**: Loads MyProperties. Grid of owned properties with “Add New” button.
 3. **Bookings**: Loads RequestList. Buttons to **Approve** or **Reject** tenant requests.
 4. **Financials**: Loads PaymentList. View incoming payments.
- **Add Property**: Opens AddPropertyForm. Uploads images and saves to DB.
- **3. SuperAdmin Dashboard (AdminDashboard.cs)**

- **Menu:**
 1. **User Management:** View all users, Delete users.
 2. **Transactions:** View system-wide payment history.
- **4. Property Details (PropertyDetails.cs)**
- **Purpose:** Detailed view of a specific house.
- **Buttons:**
 - **Book Now:** Checks if user is Tenant. Checks dates. Calls TenantService.CreateBooking().
- **Features:** Displays Start/End date pickers. Calculates total cost dynamically.
- **5. Payment Form (Payment.cs)**
- **Purpose:** Process rent payment.
- **Inputs:** Mobile Banking Provider (Bkash/Nagad), Sender Phone, Transaction ID.
- **Integration:** Contains InitSSLCommerz method which posts data to the sandbox API using HttpClient for online payment simulation.

1. ER DIAGRAM

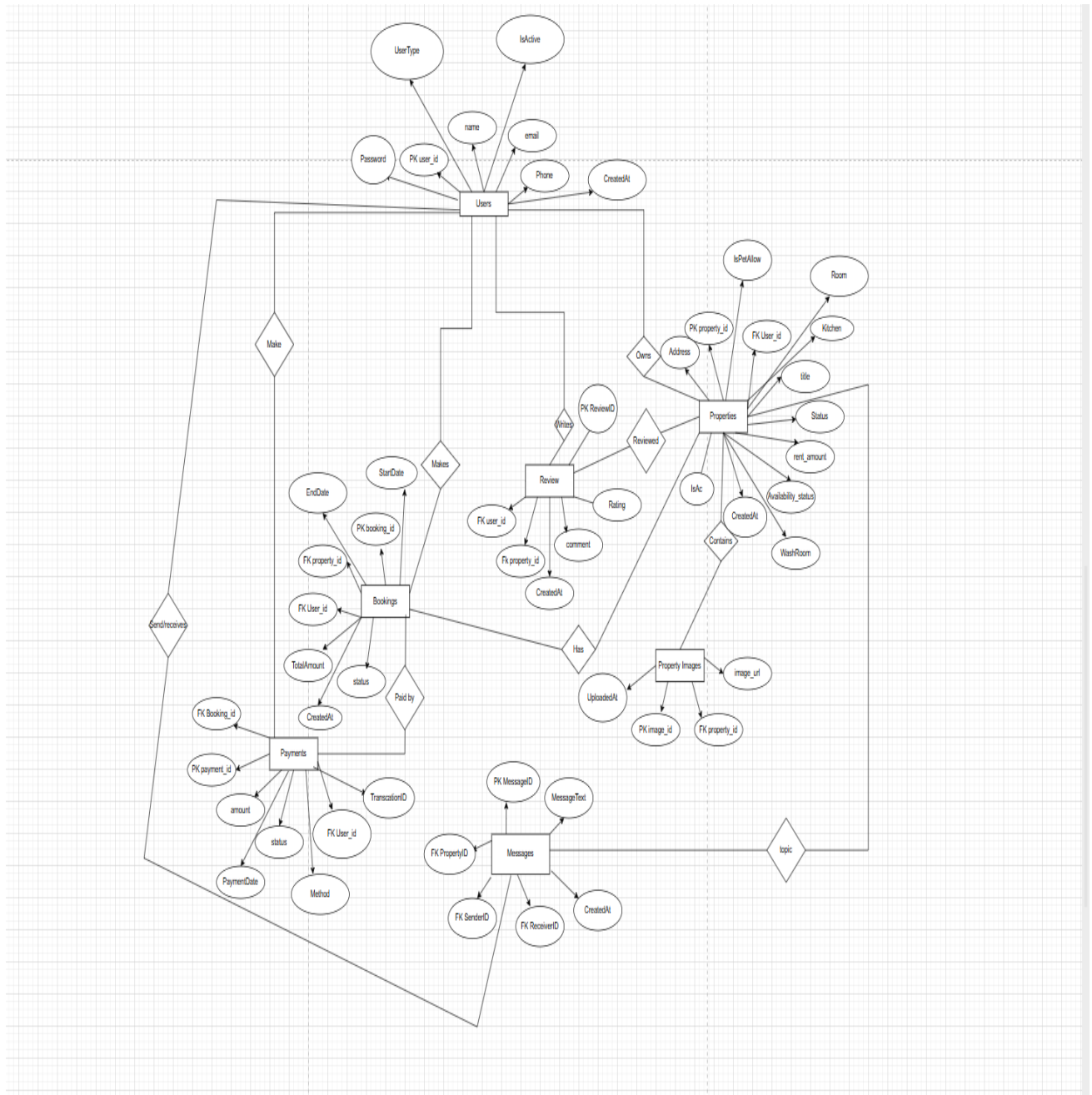


Figure:ER Diagram

1.1 Normalization

1. Relation Name: User Owns Property

UNF:

UserID, Name, Email, Phone, Password, UserType, IsActive, PropertyID, Title, Address, RentAmount, Status, Rooms, Kitchen, Washroom, IsAC, IsPetAllow, AvailabilityStatus, CreatedAt

1NF:

Multivalued attributes: Phone

UserID, Name, Password, UserType, IsActive

PropertyID, Title, Address, RentAmount, Status, Rooms, Kitchen, Washroom, IsAC, IsPetAllow, AvailabilityStatus, CreatedAt

2NF:

UserID, Email

UserID, Name, Password, UserType, IsActive

PropertyID, Title, Address, RentAmount, Status, Rooms, Kitchen, Washroom, IsAC, IsPetAllow, AvailabilityStatus, CreatedAt, **UserID(FK)**

3NF:

UserID, Email

UserID, Name, Password, UserType, IsActive

PropertyID, Title, Address, RentAmount, Status, Rooms, Kitchen, Washroom, IsAC, IsPetAllow, AvailabilityStatus, CreatedAt, **UserID(FK)**

2. Relation Name: User Makes Booking

UNF:

UserID, Name, BookingID, StartDate, EndDate, TotalAmount, Status, CreatedAt, PropertyID, Title, Address, Email,

1NF:

Multivalued attributes: Email

UserID, Name

BookingID, StartDate, EndDate, TotalAmount, Status, CreatedAt

2NF:

UserID, Email

UserID, Name

BookingID, StartDate, EndDate, TotalAmount, Status, CreatedAt, **UserID(FK)**,
PropertyID(FK)

3NF:

UserID, Email

UserID, Name

BookingID, StartDate, EndDate, TotalAmount, Status, CreatedAt, **UserID(FK)**,
PropertyID(FK)

3. Relation Name: Property Has Booking

UNF:

PropertyID, Title, Address, RentAmount,
BookingID, StartDate, EndDate, TotalAmount

1NF:

PropertyID, Title, Address, RentAmount

BookingID, StartDate, EndDate, TotalAmount

2NF:

PropertyID, Title, Address, RentAmount

BookingID, StartDate, EndDate, TotalAmount, **PropertyID(FK)**

3NF:

PropertyID, Title, Address, RentAmount

BookingID, StartDate, EndDate, TotalAmount, **PropertyID(FK)**

4. Relation Name: Booking Paid By Payment

UNF:

BookingID, StartDate, EndDate,
PaymentID, Amount, PaymentDate, Method, Status, TransactionID

1NF:

BookingID, StartDate, EndDate

PaymentID, Amount, PaymentDate, Method, Status, TransactionID

2NF:

BookingID, StartDate, EndDate

PaymentID, Amount, PaymentDate, Method, Status, TransactionID, **BookingID(FK)**

3NF:

BookingID, StartDate, EndDate

PaymentID, Amount, PaymentDate, Method, Status, TransactionID, **BookingID(FK)**

5. Relation Name: User Makes Payment**UNF:**

UserID, Name, Email,
PaymentID, Amount, PaymentDate, Method, Status

1NF:

Multivalued attributes: Email

UserID, Name

PaymentID, Amount, PaymentDate, Method, Status

2NF:

UserID, Email

UserID, Name

PaymentID, Amount, PaymentDate, Method, Status, **UserID(FK)**

3NF:

UserID, Email

UserID, Name

PaymentID, Amount, PaymentDate, Method, Status, **UserID(FK)**

6. Relation Name: User Writes Review

UNF:

UserID, Name,
ReviewID, Rating, Comment, CreatedAt,
PropertyID, Title

1NF:

UserID, Name

ReviewID, Rating, Comment, CreatedAt

2NF:

UserID, Name

ReviewID, Rating, Comment, CreatedAt, **UserID(FK)**, **PropertyID(FK)**

3NF:

UserID, Name

ReviewID, Rating, Comment, CreatedAt, **UserID(FK)**, **PropertyID(FK)**

7. Relation Name: Property Has Review

UNF:

PropertyID, Title,
ReviewID, Rating, Comment

1NF:

PropertyID, Title

ReviewID, Rating, Comment

2NF:

PropertyID, Title

ReviewID, Rating, Comment, **PropertyID(FK)**

3NF:

PropertyID, Title

ReviewID, Rating, Comment, **PropertyID(FK)**

8. Relation Name: Property Contains Images

UNF:

PropertyID,
ImageID, ImageURL, UploadedAt

Title,

1NF:

PropertyID, Title

ImageID, ImageURL, UploadedAt

2NF:

PropertyID, Title

ImageID, ImageURL, UploadedAt, **PropertyID(FK)**

3NF:

PropertyID, Title

ImageID, ImageURL, UploadedAt, **PropertyID(FK)**

9. Relation Name: User Sends Message to User (M-M)

UNF:

SenderID, SenderName,
ReceiverID, ReceiverName,
MessageID, MessageText, CreatedAt, PropertyID

1NF:

SenderID, SenderName

ReceiverID, ReceiverName

MessageID, MessageText, CreatedAt

2NF:

SenderID, SenderName

ReceiverID, ReceiverName

MessageID, MessageText, CreatedAt, **SenderID(FK)**, **ReceiverID(FK)**, **PropertyID(FK)**

3NF:

SenderID, SenderName

ReceiverID, ReceiverName

MessageID, MessageText, CreatedAt, **SenderID(FK)**, **ReceiverID(FK)**, **PropertyID(FK)**

1.2 Finalization (Final Tables)

UserID, Name, Email, Phone, Password, UserType, IsActive, CreatedAt

PropertyID, Title, Address, RentAmount, Status, Rooms, Kitchen, Washroom, IsAC, IsPetAllow, AvailabilityStatus, CreatedAt, **UserID(FK)**

BookingID, StartDate, EndDate, TotalAmount, Status, CreatedAt, **UserID(FK)**, **PropertyID(FK)**

PaymentID, Amount, PaymentDate, Method, Status, TransactionID, **BookingID(FK)**, **UserID(FK)**

ReviewID, Rating, Comment, CreatedAt, **UserID(FK)**, **PropertyID(FK)**

ImageID, ImageURL, UploadedAt, **PropertyID(FK)**

MessageID, MessageText, CreatedAt, **SenderID(FK)**, **ReceiverID(FK)**, **PropertyID(FK)**

2. UML DIAGRAM

2.1 Use Case Diagram

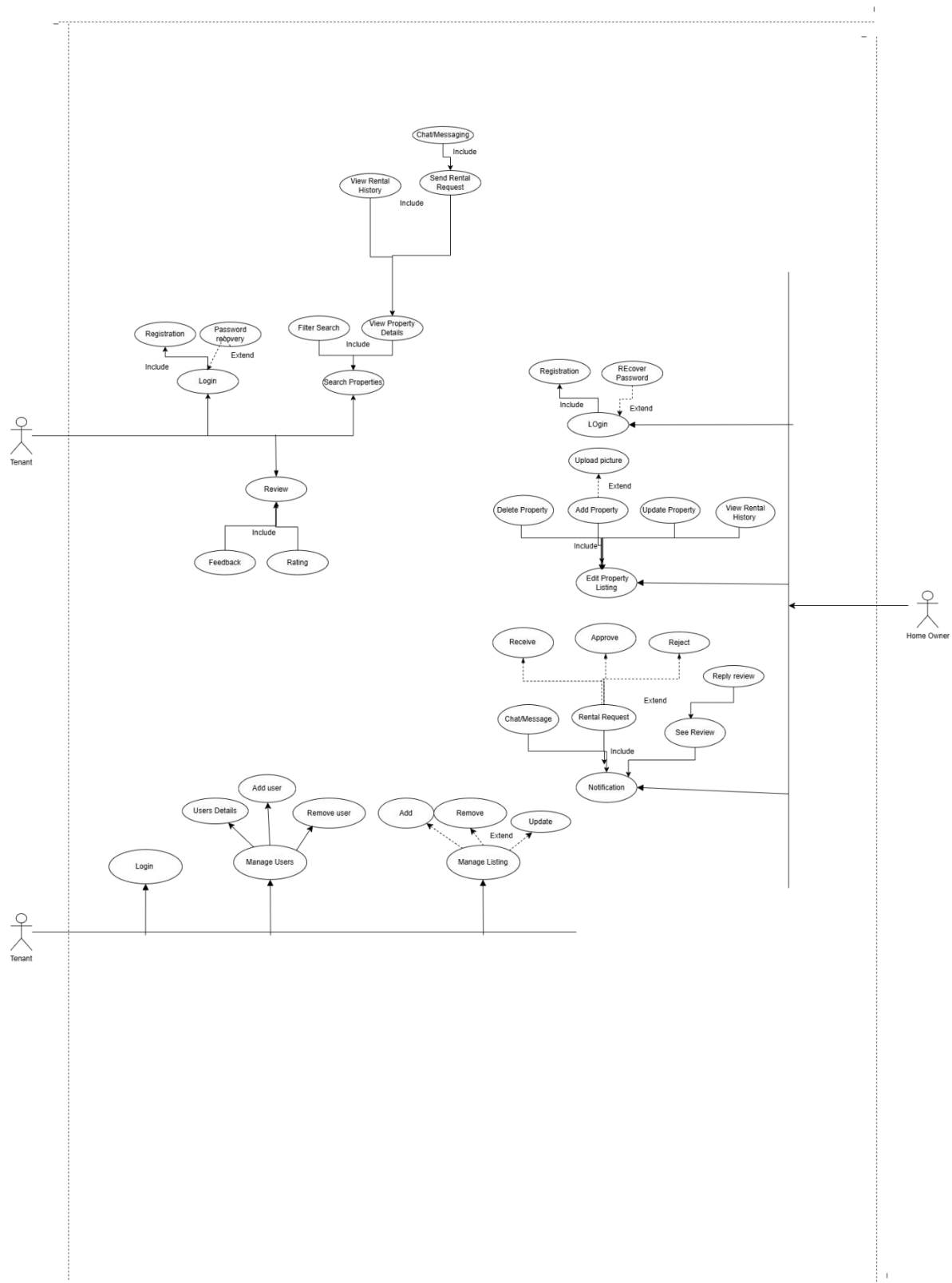


Figure: Use Case Diagram

2.2 Activity Diagram.

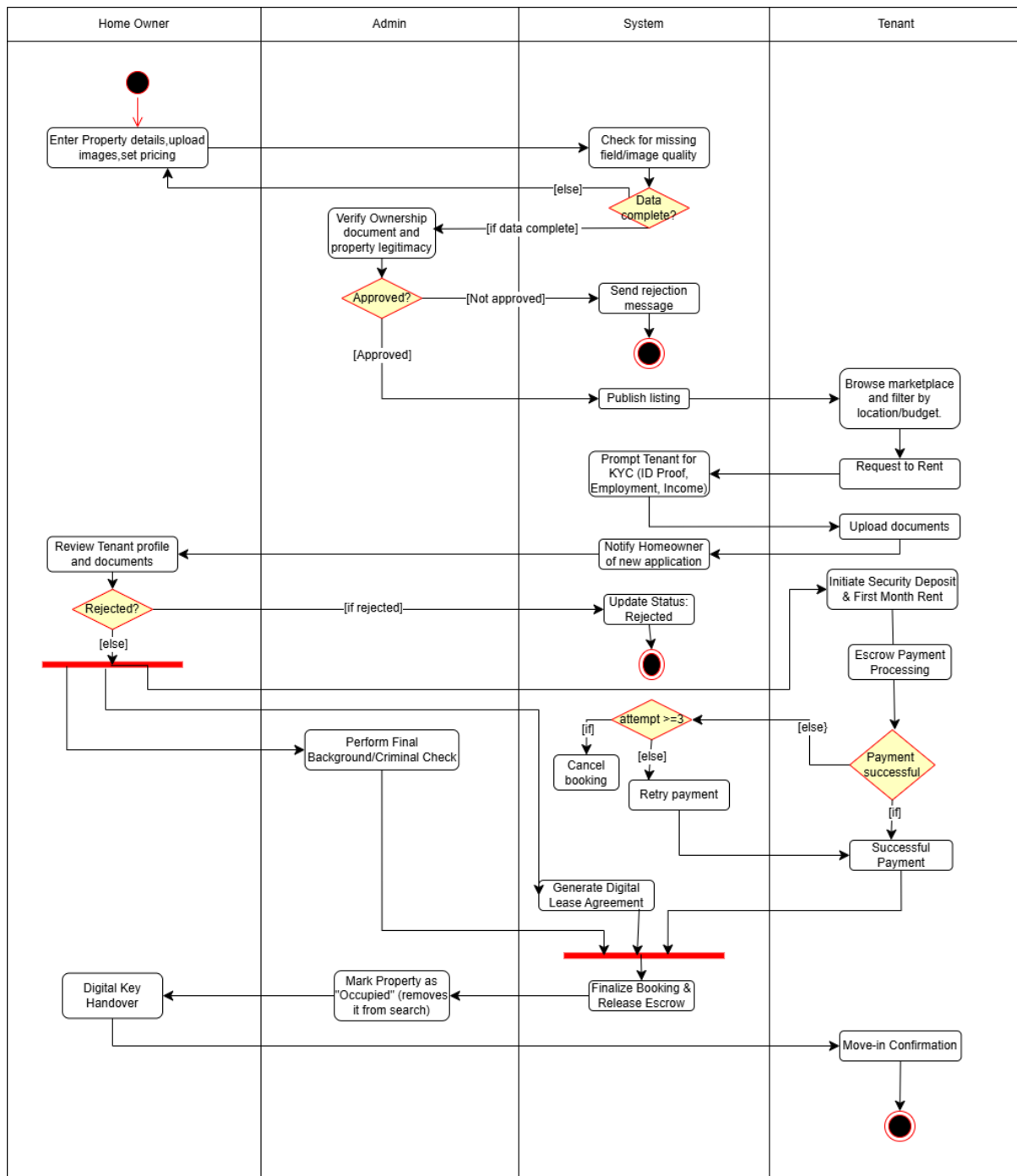


Figure: Activity Diagram

2.3 Schema Diagram

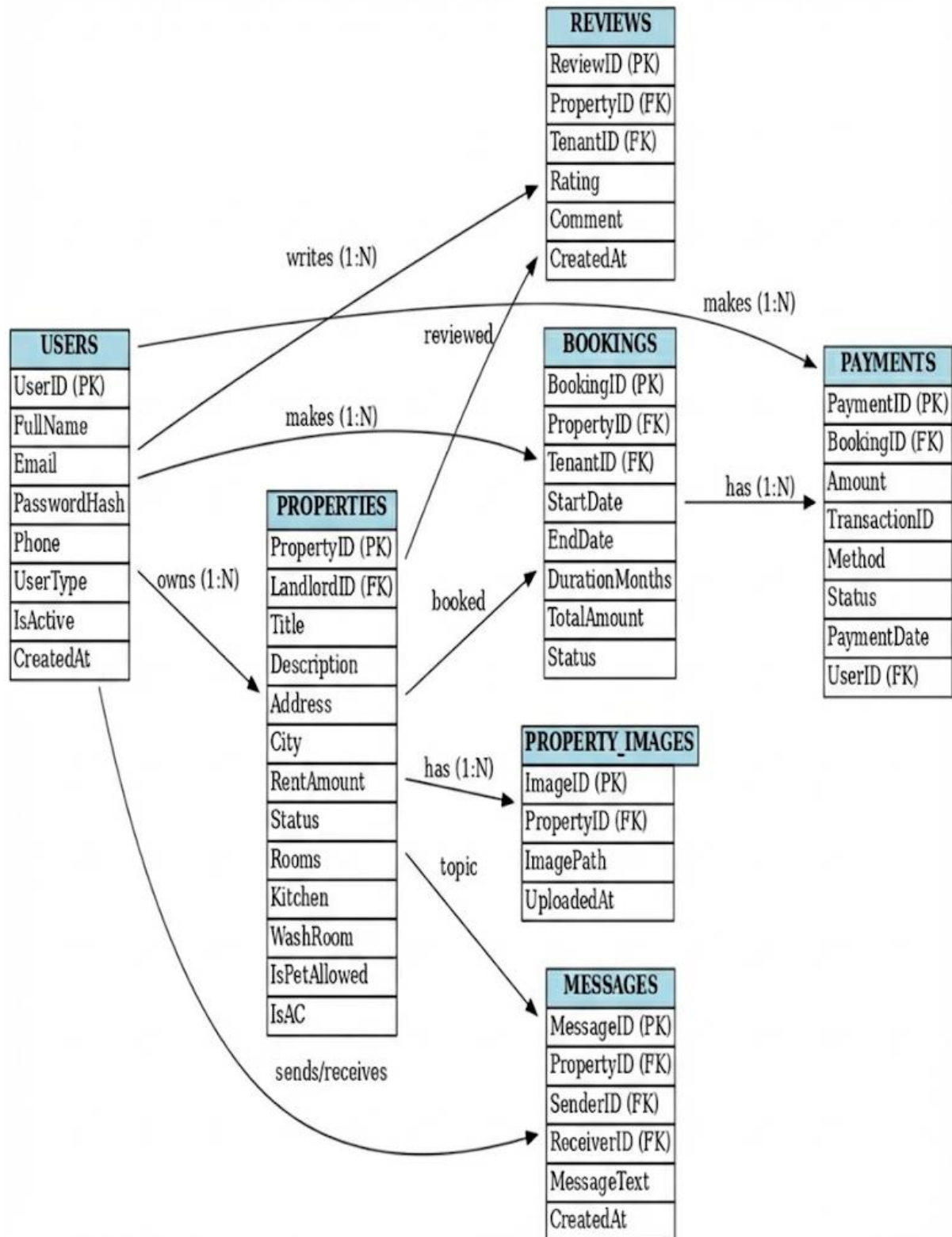


Figure: Schema Diagram

5. Project view

Landlord sign up and sign in:

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Already a member? [Log in](#)

Create Account

☐ Tenant ☒ Landlord

Full Name:

Phone Number:

Email Address:

Password:

☒ I agree to Terms of Service

[Create Account](#)

Or continue with

[Google](#) [Facebook](#)

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Welcome Back

[Google](#) [Facebook](#)

Or continue with

Email Address:

Password:

☐ Remember me [Forgot Password?](#)

[Sign in](#)

[Don't have an account? Sign up](#)

Landlord dashboard and financials:

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Dashboard Overview

Dashboard

Overview of your properties and bookings

[Add Property](#)

Total Earnings

₹30,000

Properties

2

Bookings

0

Revenue

50%

Recent Properties

Listing ID: 123456789 | Location: D-palace | Price: ₹10,000 | Status: Active

Recent Bookings

Listing ID: 123456789 | Status: Requested To Rent | Approved

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Financials

Payments & Transactions

₹30,000 Shanihan Jaman Shuchi

D-palace

Landlord bookings, messages and settings:

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Booking Requests

Tenant Bookings

Listing ID: 123456789 | Location: D-palace | Price: ₹10,000 | Status: Active

[Remove Tenant](#)

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Messages

Conversations | Property Reviews

No conversations yet.

Manage properties with ease.

The all-in-one solution for landlords, tenants, and property managers.

Settings

Account Settings

Profile Information

Full Name:

Email Address:

Phone Number:

[Save Changes](#)

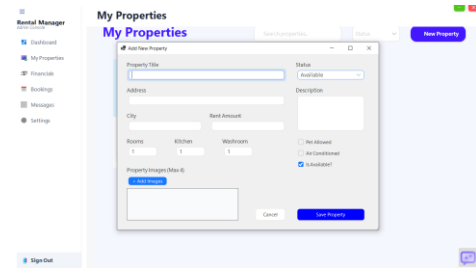
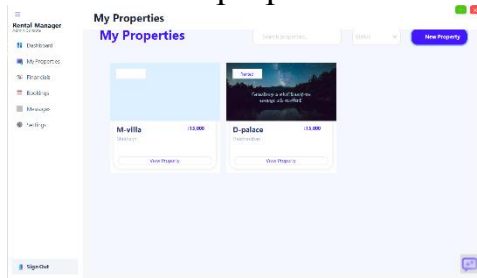
Change Password

Current Password:

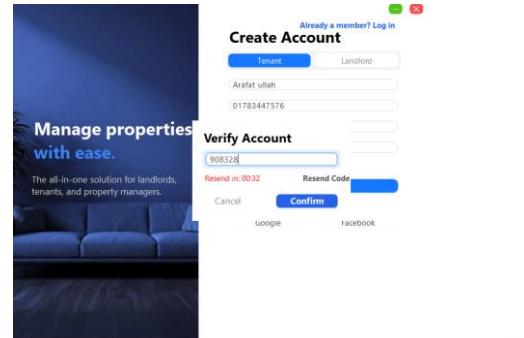
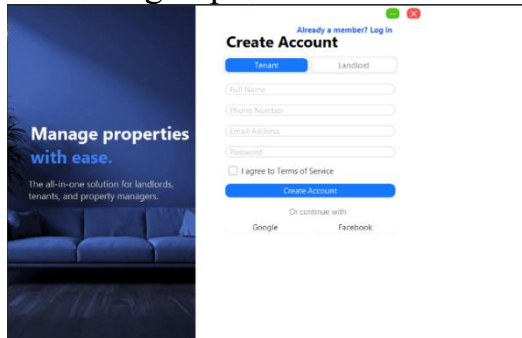
New Password:

[Update Password](#)

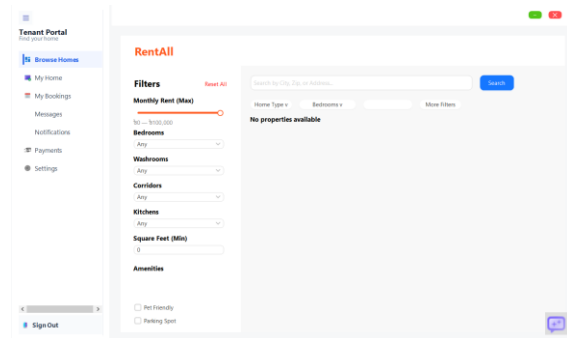
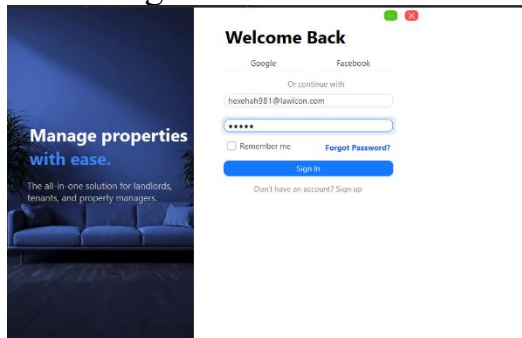
Landlord add properties:



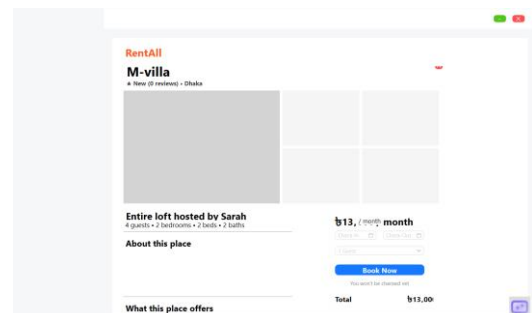
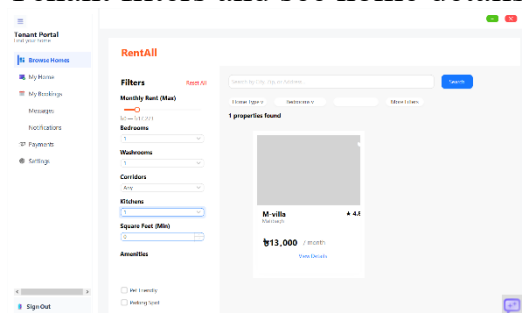
Tenant sign up:



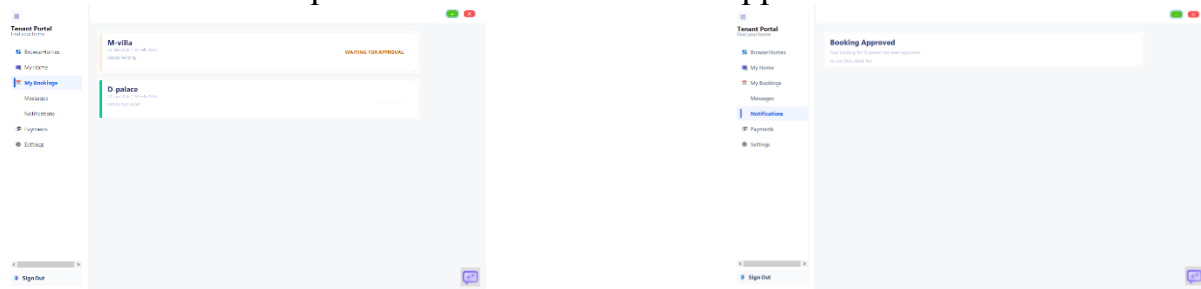
Tenant sign in:



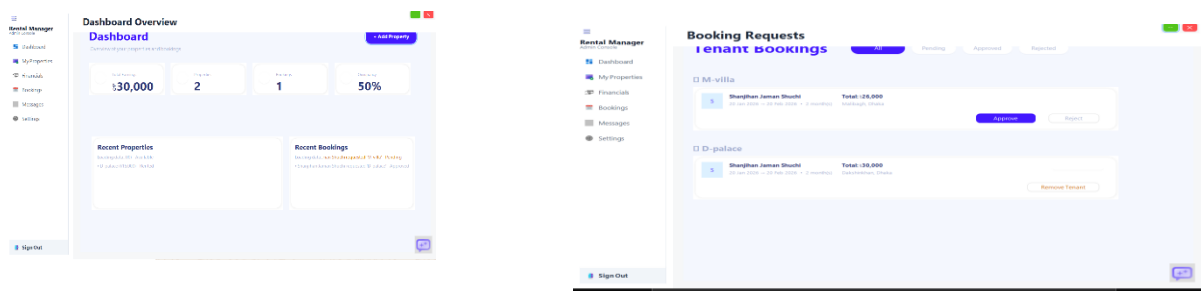
Tenant filters and see home details:



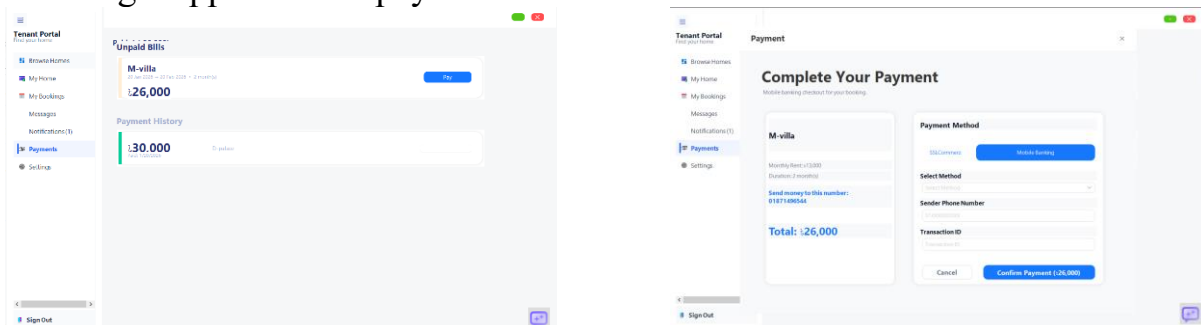
Tenant sent book request and wait for Landlord approval:



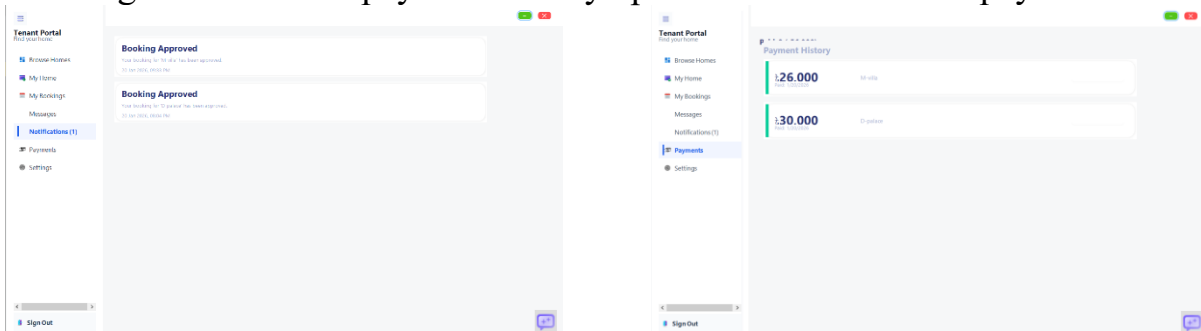
Landlord gets Recent bookings notification and approve the booking request:



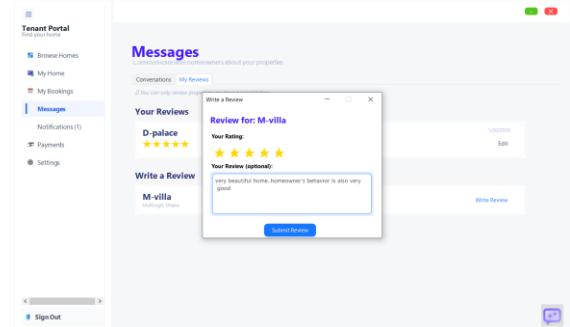
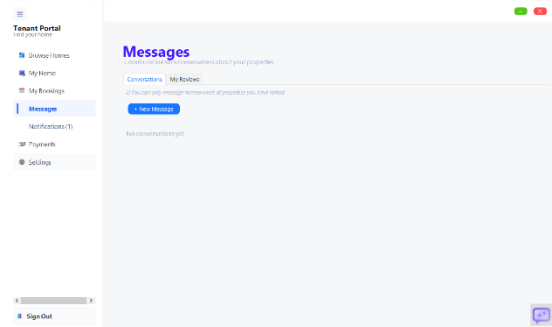
Tenant get approval and pay:



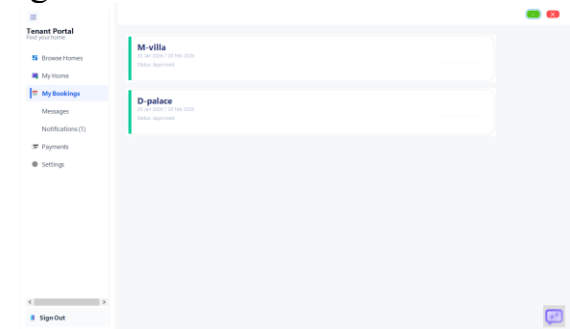
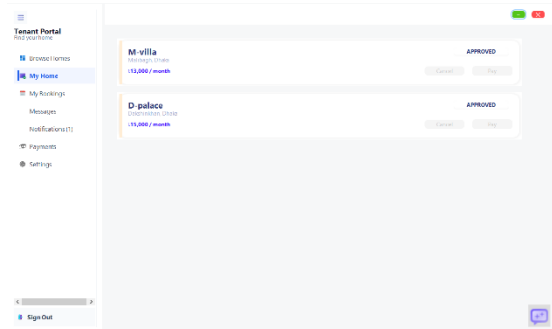
Tenant get notified and payment history updated after successful payment:



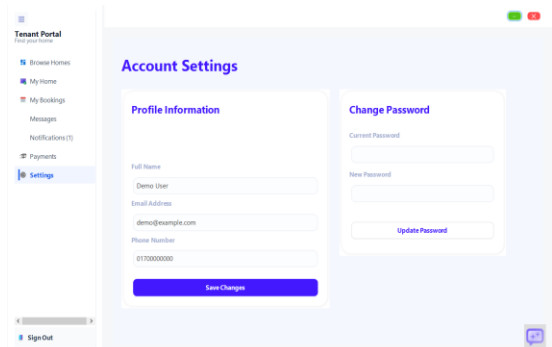
Tenant can message and give review:



Tenant can see his/her all homes and bookings:



Tenant can edit his/her account:



6.CONCLUSION

The **Rental Management System** provides a solid foundation for digitizing the traditional housing market by streamlining how tenants and landlords interact. By moving away from manual tracking and adopting a secure, layered architecture, the platform ensures that property management is both efficient and reliable. From secure password hashing with **BCrypt** to real-time status updates for bookings, the system addresses the core needs of modern rental management.

Looking Ahead: Future Work

While the current version is fully functional, there is always room to grow. Here are a few directions we plan to take the project in the future:

- **Mobile Integration:** While the system is currently a robust desktop application using WinForms, developing a mobile companion app would allow tenants and landlords to manage bookings on the go.
- **Automated AI Service:** An Automated AI-Based Home Rental System is a smart platform that helps users find suitable houses using AI recommendations. It also helps owners set proper rent, detects fake listings, and makes the renting process faster, easier, and safer.
- **Advanced Analytics:** For Landlords and SuperAdmins, we plan to expand the dashboard statistics to include visual charts and trends, such as monthly revenue growth and seasonal demand heatmaps.
- **Favorite Properties:** As outlined in our expansion plan, we intend to implement a "Favorites" feature, allowing tenants to save listings they are interested in for quick access later.
- **Automated Rent Reminders:** By leveraging the existing **Resend API** integration, we can automate monthly email notifications to remind tenants when their rent is due, further reducing the administrative burden on landlords.
- **Document Management:** Adding a feature to upload and store digital lease agreements or NID copies directly within the system would make the verification process even more secure and organized.

7.Reference:

- **Language:** C# (.NET Core / .NET Framework)
- **UI Framework:** Windows Forms (WinForms) with **AntdUI**
- **Database:** Microsoft SQL Server (LocalDB / SQLEXPRESS)
- **Architecture:** Layered Architecture (UI → Service → Repository → Database)
- **Authentication:** Custom Session Management with **BCrypt Password Hashing**
- **Email Service:** **Resend API** (Transactional emails, OTP verification, alerts)
- **Payment Simulation:** SSLCommerz Sandbox (via HttpClient)

Rental System UI is a C# WinForms-based desktop application that manages property rentals using a layered architecture. It enables tenants to book properties, landlords to manage listings and bookings, and admins to oversee users and transactions. The system uses SQL Server, secure BCrypt authentication, and email-based verification.